EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Global Claims and Order Processing Enterprise (Platform-Neutral Case Study)

Name: RAMANATHAN S **Roll No:** 241401076

1. Aim

To examine, through a case study, the limits that low-code platforms as a category reach when they are applied to a complex enterprise system, rather than judging any single vendor. The scenario is a company that handles customer orders and warranty claims in many countries, and the study ends with suggestions for keeping those limits under control.

The case study asks three things: where low-code clearly helps, where it begins to struggle as scale and complexity increase, and how a hybrid design can keep the speed of low-code while covering its weak points.

2. Procedure

Step 1: Set the scenario. A global company was described that takes orders and claims through web and mobile portals, serves agents, finance staff and administrators, runs a legacy ERP or mainframe, uses payment and logistics partner APIs, and must follow tax, privacy and audit rules that differ by country.

Step 2: Derive the requirements. The scenario was split into six requirement areas: scale, business rules, integration, security and compliance, tailored screens, and long-term upkeep (Section 3.2).

Step 3: Draw the integration map. Every link to the platform was drawn and coloured by how well a typical low-code platform supports it: well supported, possible with effort, or strained (Section 3.1).

Step 4: Analyse the limits. Each common limitation was written down with its effect on this company and a way to reduce it (Section 3.3).

Step 5: Show the strain. A conceptual curve and an indicative suitability chart were prepared to show where low-code fits and where it does not (Sections 3.4 and 3.5).

Step 6: Choose and recommend. A decision flow was drawn for picking between low-code, hybrid and custom development (Section 3.6), followed by a recommended hybrid architecture (Section 3.7).

Note: this is an analytical case study. Actual limits such as run time, record counts or API calls vary by platform and plan, so no figures are quoted; the documentation of the chosen platform should be checked.

241401076

3. Output

3.1 Enterprise scenario and integration map

The map places a generic low-code platform at the centre of the company. Built-in platform services and ready-made connectors are well supported (green). Payment and logistics APIs, and the country-by-country tax, privacy and audit rules, are possible with effort (amber). The legacy ERP or mainframe, and fraud scoring or demand analytics, are strained and usually need custom code (red).

3.2 Enterprise requirements

Requirement	Why it is complex
Scale	Orders and claims arrive from many regions, with sharp peaks during sales seasons.
Business rules	Tax, currency, warranty and claim rules differ by country and change frequently.
System integration	A legacy ERP or mainframe, payment and logistics APIs, and company-wide sign-in.
Security and compliance	Privacy and financial rules require audit trails, access control and local data storage.
Tailored screens	Customers, agents and finance staff each need a different interface and language.
Long-term upkeep	Many teams change the system over the years, and the company may switch platform later.

3.3 Limitation analysis

Area	Limitation of low-code	Mitigation
Performance at scale	Shared, vendor-managed runtime with usage caps set by plan.	Tune queries, avoid heavy loops, push bulk jobs to external services.
Complex logic	Built-in scripting suits routine rules, not advanced algorithms.	Place the logic in a custom service and call it by API.
Integration	Ready connectors cover popular tools; legacy interfaces need extra work.	Add an API gateway or middleware between platform and legacy systems.
Customisation	Interface follows fixed components and platform conventions.	Use custom components or a separate front end over the platform API.
Compliance & residency	Hosting regions and encryption choices depend on the vendor.	Check vendor certifications, restrict access, store sensitive data securely.
Testing & changes	Version history and automated release tools are often basic.	Use test environments, peer review and a written release routine.
Sprawl	Easy building lets each team create its own unmonitored apps.	Keep a central app register, shared standards and a named owner.
Vendor lock-in	Apps and workflows are bound to one platform and hard to export.	Keep core logic in APIs and keep data exportable.

241401076

3.4 Where low-code starts to strain

The curve shows a large benefit from low-code for simple needs. As complexity, transaction volume and integration demands increase, that benefit falls, while custom code stays fairly steady. The shaded hybrid zone marks where using both together is usually best.

3.5 Suitability of low-code by requirement

Quick first release, order and claim forms, and approvals score highest (5/5). Peak-season performance, audit control, testing, and country-specific screens score moderately (3/5). Legacy ERP integration, fraud checks, and portability score lowest (1-2/5).

3.6 Decision flow for choosing the approach

The flow asks three questions: is the need a routine form, approval or report; does it require complex algorithms or high load; and must it link to legacy or strictly regulated systems? The answers lead to pure low-code, custom code, a hybrid design, or low-code with connectors.

3.7 Recommended hybrid architecture

The low-code platform keeps the forms, approvals and screens, and relies on built-in services for sign-in and messages. An API gateway or middleware layer links it to custom services for fraud and pricing, to the legacy ERP and payment systems, and to a data warehouse and audit store. Monitoring, role-based access and governance cover every layer.

4. Final Result

The case study found that low-code is very effective for the routine parts of an enterprise system: forms, approvals, notifications and ecosystem integrations. It delivers quickly with little code, whichever platform is chosen.

For a complex, global use case, the main limitations were performance at scale, complex logic, legacy and industry integration, restricted customisation, compliance and data-residency control, weak testing and change control, app sprawl and vendor lock-in.

The recommended solution is a hybrid approach: keep low-code for forms, workflows and screens, and add an API or middleware layer with custom services for heavy logic, legacy integration and analytics, under clear governance.

241401076